

IA Aplicada com LLMs

Aula 06 — Bancos vetoriais

Onde paramos

Os vetores moram numa **matriz de numpy** que morre com o processo.

A nota 02 defendeu isso com número, e continua certa para 28 chunks.

Mas e quando forem dez milhões?

Roteiro

1. O que um banco vetorial faz
2. Por que a busca **não** é exaustiva
3. Quais tempos — medidos
4. Como se **insere**
5. Como se **busca**, em quatro situações
6. Como se **lê** o resultado
7. Os bancos que existem

Objetivos

- **Explicar** as três funções: armazenar, indexar, buscar
- **Distinguir** busca exata de busca aproximada, e o preço de cada uma
- **Inserir e buscar**, com e sem filtro de metadado
- **Ler** o resultado — e saber que o banco **nunca** diz "não tenho isso"
- **Justificar** o uso de um banco pelo argumento certo para a escala

Parte 1

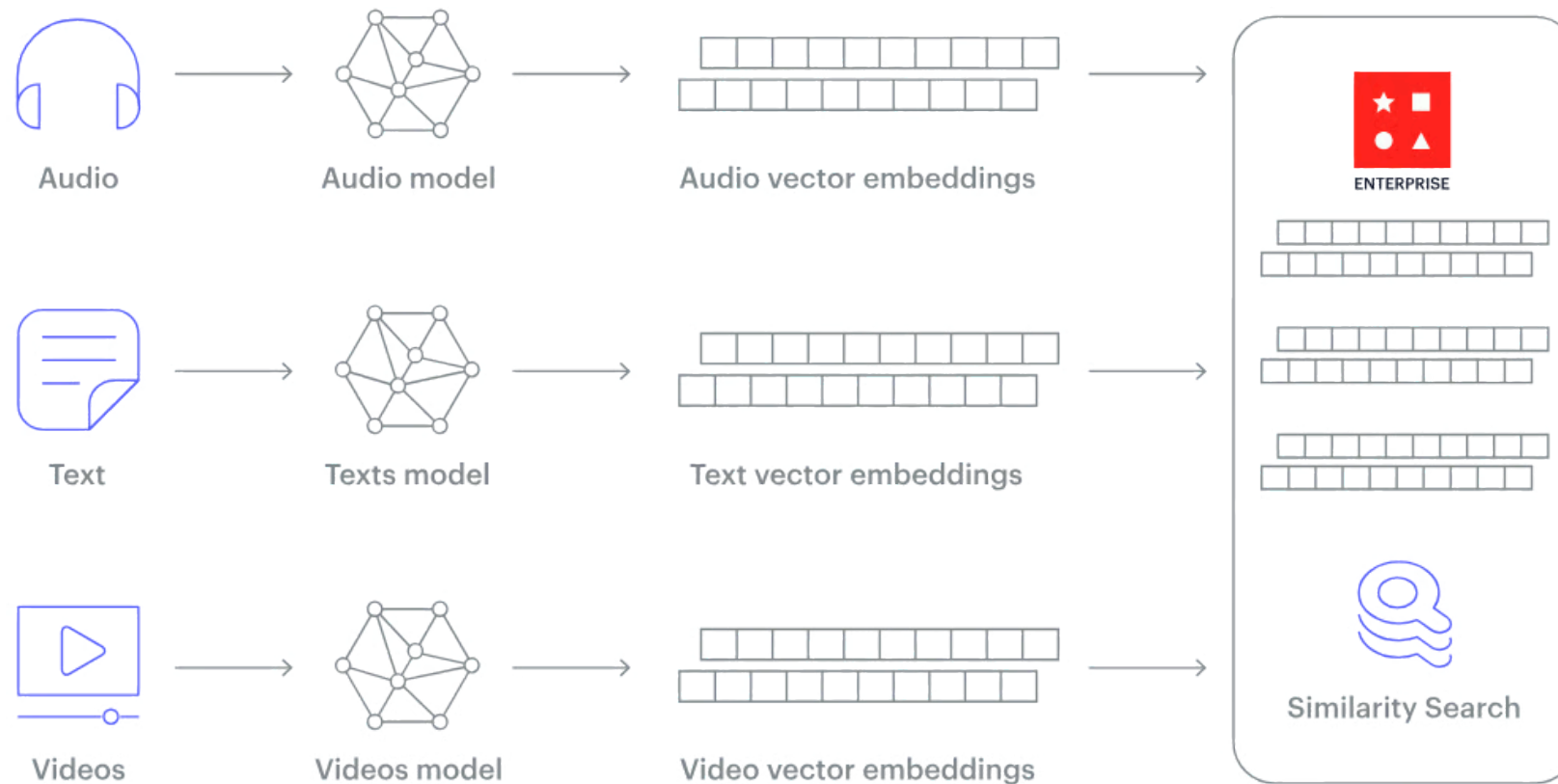
O que ele faz

Três funções

Função	O que faz	numpy já fazia?
Armazenar	vetor + metadados, sem recalcular	em parte
Indexar	estrutura que acelera a busca	não
Buscar	compara consulta × indexados	sim, exaustivamente

A diferença real é a linha do meio.

E o vetor pode ser de qualquer coisa



Fonte: DataCamp, *The Top 5 Vector Databases*

O banco não sabe o que o vetor representa. Ele compara **números**.

Parte 2

Por que a busca não é exaustiva

O custo do óbvio

numpy
aproximada

compara com TODOS
navega vizinhos

$O(n)$
 $\sim O(\log n)$

ANN — *approximate nearest neighbour*: recupera vetores parecidos sem varrer o conjunto inteiro. Em troca, **pode errar o vizinho mais próximo**.

O nome é honesto. A pergunta de projeto não é "exato ou aproximado" — é **quanto recall por quanto de latência**.

Os algoritmos que aparecem

	Ideia
HNSW	grafo em camadas: ligações longas em cima, vizinhança densa embaixo
IVF	particiona o espaço, varre só as células próximas
LSH	hash que joga vetores parecidos no mesmo balde
PQ	comprime o vetor num código curto que preserva distância relativa

HNSW é o mais comum hoje, e é o que o Chroma usa.

Reter a intuição: **em vez de comparar com todos, navega-se por vizinhos.**

A métrica é declarada, não adivinhada

```
colecão = cliente.get_or_create_collection(  
    name="regulamento", metadata={"hnsw:space": "cosine"})
```

cosseno	ângulo — o desta aula
produto escalar	ângulo e magnitude
euclidiana	a reta entre as pontas

A métrica entra no **carimbo da versão**. Trocá-la muda o `recall` sem que ninguém mude o código.

Parte 3

Quais tempos

Medido, não afirmado

Mesma pergunta · mesmos 28 chunks · 200 repetições

numpy, comparando com TODOS	0,040 ms
Chroma	0,472 ms

O banco é dez vezes mais lento.

E não há defeito nisso: ele paga serialização e travessia de grafo para economizar 28 comparações que custam quase nada.

O ganho está na escala

Escala	Exaustivo	Veredito
28 (esta aula)	instantâneo	banco é desnecessário
100 mil	ainda viável	começa a compensar
10 milhões	inviável	obrigatório

A pergunta não é "banco ou numpy". É quantos vetores você tem.

E, nesta aula, o argumento não é velocidade — os números proíbem. É persistência.

Parte 4

Inserir

Quatro listas paralelas

```
colecacao.add(  
    ids=[f"{i:03d}" for i in range(len(chunks))],      # a chave  
    documents=[c["texto"] for c in chunks],            # o texto original  
    metadatas=[{"artigo": c["id"],                      # o que dá para filtrar  
                 "numero": numero_do_artigo(c["id"])} for c in chunks],  
    embeddings=vetores.tolist(),                       # o vetor, PRONTO  
)
```

- `ids` — reinserir o mesmo id **substitui**
- `metadatas` — é por aqui que se **filtra**
- `embeddings` — calculado **fora** do banco

E isso é decisão de projeto

Vários bancos se oferecem para **embutir por você**, a partir do texto.

Aceitar esconde qual modelo produziu os vetores — e no dia em que ele mudar, o índice inteiro muda de significado **sem aviso**.

É o mesmo princípio que fez o `embedding.py` ser a única porta da aula para a API.

Parte 5

Buscar

[1] A busca comum

```
r = colecao.query(query_embeddings=[v.tolist()], n_results=3)
```

"qual o teto de refeição em viagem?"

Art. 4º §1º	distância 0,2356
Art. 4º §2º	distância 0,2402
Art. 6º §2º	distância 0,2585

[2] A busca com filtro

```
r = colecao.query(query_embeddings=[v.tolist()], n_results=3,  
                  where={"numero": 9})
```

Art. 9º §2º	distância 0,3269
Art. 9º §1º	distância 0,3284
Art. 9º §3º	distância 0,3327

O filtro roda antes da comparação de vetores. Faixa de valor, data e identificador se decidem aqui — com `==` e `<=`, não com cosseno.

Filtro + vetor na mesma consulta tem nome de mercado: **busca híbrida**.

[3] O texto procurando a si mesmo

```
Art. 1º          distância 0,0000
```

O piso de baixo, e um **teste de sanidade**.

Se um texto do corpus não se encontra em distância zero, alguma coisa está errada na indexação — e o erro não dá exceção.

[4] A pergunta fora do domínio

"qual o índice de reajuste da tabela de fretes marítimos?"

Art. 5º §1º distância 0,2586

O regulamento não fala de fretes. E a distância **não é alta** — é quase a mesma da pergunta pertinente.

O banco nunca diz "não tenho isso".

Ele devolve os **k** mais próximos. Sempre.

Parte 6

Ler o resultado

Listas paralelas, e o índice [0]

```
r["ids"][0]          ['008', '009', '016']  
r["distances"][0]    [0.2356, 0.2402, 0.2585]  
r["metadata"][0]     [{'artigo': 'Art. 4º §1º'}, ...]  
r["documents"][0]    os textos, na mesma ordem
```

O `[0]` existe porque a API aceita **várias consultas** numa chamada.

As quatro listas são **paralelas** — percorrer com `zip` não é estilo, é o único jeito correto.

E o número é distância

$$d = 1 - \cos(\theta)$$

Situação	Distância	Cosseno
texto igual a si mesmo	0,0000	1,0000
pergunta pertinente	0,2356	0,7644
fora do domínio	0,2586	0,7414

0,023 é toda a margem entre uma pergunta que o corpus responde e uma que ele não responde.

Qualquer limiar chutado cai fora dessa faixa. É a nota 05.

Parte 7

Os bancos que existem

Sete nomes, uma categoria

	Chroma	embutido, diretório local — o desta aula
	Pinecone	gerenciado, sem versão para rodar em casa
	Weaviate	aberto, com módulos de embedding integrados
	Qdrant	aberto, em Rust, filtro avançado no payload
	Milvus	aberto, distribuído, bilhões de vetores
	pgvector	extensão do PostgreSQL — não é um banco novo
	OpenSearch	motor de busca com suporte a vetor

Duas observações que poupam discussão

A penúltima linha é a mais subestimada. Se a aplicação já roda sobre PostgreSQL, o `pgvector` evita operar um sistema a mais — e a maioria dos projetos não tem vetores que justifiquem esse sistema a mais.

A API muda, o modelo não. Os sete guardam id, documento, metadado e vetor; todos indexam com alguma variante de ANN; todos devolvem os `k` mais próximos com uma medida de proximidade.

Aprender um é aprender a categoria.

O que fica

- Um banco acrescenta **índice**, não armazenamento
- **ANN** troca exatidão por latência — e o nome avisa
- Em 28 vetores o banco **perde**; o ganho é de escala e de persistência
- **O filtro de metadado** resolve o que o vetor não representa
- O retorno é **distância**, e **baixo é bom**
- O banco **nunca** diz "não tenho isso"

Próxima nota: quem diz isso é o **portão** — e os trechos viram resposta.